Hackathon Problem Statement: Enhancing RAG for Code Intelligence

Palacode Narayana Iyer Anantharaman

November 13, 2025

1 Background

Large Language Models (LLMs) can be significantly enhanced using Retrieval-Augmented Generation (RAG), where contextual information is retrieved from an external knowledge source before answering a query.

In this hackathon, participants will enhance a RAG system built for code understanding. The system retrieves relevant Python and Rust code segments, along with their summaries, from a vector database and uses them to answer developer-style questions about the repository.

2 Repository Setup

Clone the latest version of Andrej Karpathy's `nanochat` repository using:

```
git clone https://github.com/karpathy/nanochat.git
cd nanochat
```

This repository contains the source code that forms the knowledge base for your RAG system. Starter scripts for ingestion, retrieval, and question answering will be provided.

3 Objectives

Enhance the RAG pipeline with the following features:

Task 1. **Traceability:** Modify the answer-generation step so that each response explicitly references the file(s) from which it draws information. Use metadata fields like `source` for this purpose.

Task 2. **Conceptual Reasoning:** Improve performance on higher-level conceptual questions (e.g., "How is the tokenizer implemented?") by leveraging file-level summaries present in metadata.

Task 3. **Query Rewriting (Bonus):** Implement a preprocessing step that rewrites vague or underspecified user questions into clearer, more informative ones before retrieval.

4 Working with LangChain Documents

Each code file is stored as one or more `Document` objects:

```
Document(
    page_content = "<chunk of source code>",
    metadata = {"source": "path/to/file.py"}
)
```

You can add new metadata fields as follows:

```
doc.metadata["summary"] = summary_text
doc.metadata["language"] = "python"
doc.metadata["chunk_id"] = i
```

Metadata can then be used to:

- Filter documents:

  ```
  retriever.get_relevant_documents(query, filter={"language": "python"})
  ```

- Cite files in answers for traceability.
- Support conceptual answers using file-level summaries.

5 Metadata in Real Applications

Metadata plays a vital role in real-world RAG pipelines:

- **Traceability:** Enables users to trace answers to specific files or modules, e.g., "Defined in `tasks/mmlu.py`".
- **Filtering:** Retrieve only documents matching specific criteria such as programming language, directory, or commit version.
- **Attribution:** Ensures reliable citations in generated reports or documentation.
- **Versioning and Temporal Queries:** Enables users to query code differences across releases.

6 Tutorial: Query Rewriting for Improved Retrieval

Query rewriting enhances retrieval when user questions are vague, incomplete, or too conversational. It reformulates them into precise, code-aware questions.

Why Query Rewriting?

Retrievers depend on lexical or semantic similarity between a query and the corpus. Natural language queries like "How is training handled?" may not match code identifiers such as `train.py` or `Trainer`. Query rewriting bridges this gap.

Techniques

- **Rule-based rewriting:** Expand queries using known keywords or code context.
- **LLM-based rewriting:** Use a prompt-driven LLM call to clarify or expand user queries.
- **Hybrid methods:** Combine rules with an LLM to balance determinism and adaptability.

Example 1:

Original Query: "How is the tokenizer implemented?" **Rewritten Query:** "Explain the algorithm and functions that define the tokenizer implementation in `nanochat/tokenizer.py` or related Rust files, including BPE logic, pre-tokenizers, and vocabulary loading."

Improvement: Adds code-specific context ("BPE logic", "tokenizer.py") to focus retrieval on the right modules.

Example 2:

Original Query: "How is the model trained?" **Rewritten Query:** "Describe the training pipeline defined in `train.py`, including the dataset loading process, optimizer configuration (AdamW), and training loop (forward and backward passes)."

Improvement: Clarifies intent and ensures retrieval matches files related to model training.

When to Use Query Rewriting

- When the query is abstract or lacks domain-specific terminology.
- When multiple related modules contribute to the answer.
- When user phrasing does not match the repository's code vocabulary.

7 Evaluation Criteria

Each team will be evaluated according to the following rubric:

Criterion	Weight (%)	Description / Expected Outcome
Traceability in Answers	30%	Answers clearly reference or cite file paths from metadata. Example: "Implemented in `tasks/mmlu.py`".
Conceptual QA using Summaries	30%	File summaries are effectively integrated into responses to improve design and architecture explanations.
Query Rewriting (Bonus)	20%	The rewritten queries lead to improved retrieval quality and more accurate conceptual answers.
Code Quality and Modularity	10%	Clean, modular, and well-documented code.
Demonstration and Explanation	10%	Clear demo with sample queries and comparative performance evidence.

8 Manual Evaluation: Question Set

Participants will answer 20 evaluation questions covering both code-level and conceptual understanding of the `nanochat` repository. Answering these manually or using the enhanced RAG system will be used to evaluate performance.

Code-Level Questions

C1. Explain the function `def forward(self, x): ...` in the class `MLP` inside `nanochat/.../model.`

C2. Identify the method where rotary embeddings are applied (hint: look for `apply_rotary_emb`) and describe how it integrates into the attention mechanism.

C3. In the file `nanochat/adamw.py`, describe how the custom AdamW optimizer differs from standard PyTorch's `torch.optim.AdamW`.

C4. For the file `scripts/tok_train.py`, write three unit test cases that verify vocab size, number of special tokens, and compression ratio.

C5. What does the function `repeat_kv(x, n_rep)` in `nanochat/.../model.py` do? Provide example input/output shapes.

C6. Describe the role of the file `nanochat/dataset.py` — how are shards generated and how is the `-n` flag used?

C7. In `rustbpe/.../tokenizer.rs`, explain how the vocabulary is loaded from file to encoding step.

C8. For the script `speedrun.sh`, outline the key steps from setup to inference.

C9. In the SFT or RL task modules (e.g., `SimpleSpelling`, `MMLU`), explain the task interface and its key methods (inputs, outputs, evaluation).

C10. In `nanochat/report.py`, describe how the "CORE" metric is computed and what data it uses.

Conceptual and Architecture Questions

A11. How is tokenization implemented in `nanochat`? Explain the algorithm including byte-level BPE, special tokens, and conversation formatting.

A12. Outline the pretraining → midtraining → SFT → inference pipeline and describe the transitions between them.

A13. Why did the project author build a custom Rust tokenizer instead of using Hugging Face or `tiktoken`?

A14. How does `nanochat` support multi-device training (e.g., 8×H100)? Which scripts enable distributed setup?

A15. How does summary metadata help answer questions like: "Which files define evaluation metrics vs model architecture?"

A16. What is the purpose of the "eval bundle" and why is it kept separate from the main dataset?

A17. How do chunking and summary metadata improve retrieval in a code-RAG scenario?

A18. What are the trade-offs between prepending summaries into `page_content` and keeping them only in metadata?

A19. How could `nanochat` be extended to support multimodal (vision + text) inputs, and which parts of the pipeline would need modification?

A20. How would you find and verify the maximum sequence length (`block_size`) for pretraining in the default configuration?

9 RAG Performance Expectations

Code-Level Queries

- Focus on precise retrieval from relevant files.
- Include exact function signatures or code blocks.
- Always cite file paths via metadata.

Conceptual Queries

- Incorporate summaries from metadata to provide context.
- Use query rewriting to expand ambiguous or short questions.
- Aggregate multi-file context into coherent answers.

Improving Accuracy and Recall

- Combine semantic and keyword retrieval.
- Prepend summaries to embeddings for richer representation (optional, cost trade-offs).
- Apply LLM-based query rewriting.
- Adjust chunk sizes for better contextual coverage.

10 Deliverables

Participants must submit the following by the end of the 4-hour session:

1. **Code artifacts:**
   - Modified ingestion/retrieval scripts that show where `Document` metadata is extended (e.g., adding `summary`, `source`, `language`).
   - Modified answer-generation script that produces traceable answers (inline citations or a `Sources:` section).
   - Query rewriting module (if implemented) with its rewrite function and examples.
2. **README.md** that includes:
   - Short description of the approach and design decisions.
   - Instructions to run the pipeline locally.
   - Example queries and sample outputs (before/after rewriting if implemented).
3. **Demo notebook or script** demonstrating:
   - Ingest (or show pre-ingested data).
   - A few sample queries (at least one code-level and one conceptual).
   - Output showing traceability (file paths) and usage of summaries.
4. **Optional:** Short note (one paragraph) describing further improvements and limitations.

11 Suggested Timelines (4 hours)

The following timeline is recommended for smooth progress:

- **0:00 – 0:20 (20 minutes)** – Setup and onboarding
  - Clone repo, inspect starter scripts, verify vector DB access (or use precomputed embeddings).
  - Run a couple of baseline queries to understand current behavior.
- **0:20 – 1:40 (1 hr 20 min)** – Implement Traceability

  - Add/ensure `source` and `summary` fields are present in each `Document`.
  - Modify prompt/template to force a `Sources:` section and inline citations.
  - Test with code-level queries and verify file citations appear.

- **1:40 – 2:40 (1 hr)** – Integrate Summaries for Conceptual QA
  - Add summaries into retrieval context or as metadata displayed in prompts.
  - Test conceptual queries (e.g., tokenizer, training pipeline) and verify improved answers.

- **2:40 – 3:40 (1 hr)** – Optional: Implement Query Rewriting Tuning
  - Implement LLM-based or rule-based rewriting.
  - Compare retrieval before/after rewriting and adjust prompt or heuristics.

- **3:40 – 4:00 (20 minutes)** – Final testing, demo, and submission
  - Run 3 test queries (one code-level, one conceptual, one ambiguous).
  - Prepare a short demo and update README with instructions.

12 Expected Outcome

By the end of this hackathon, participants should have:

- A traceable RAG system citing relevant source files.
- Enhanced conceptual reasoning using summaries in metadata.
- Optional query rewriting to improve retrieval and clarity.
- Demonstrated understanding of how metadata and rewriting boost RAG performance.

Appendix: Query Rewriting Implementation Template

LLM-Based Rewriting Example (Python Pseudocode)

```
def rewrite_query(original_query):
    prompt = f"""
    You are a system that rewrites vague developer questions
    into explicit, code-aware queries that mention likely files,
    functions, or keywords.

    Example 1:
    Q: How is the tokenizer implemented?
    A: Explain the tokenizer implementation in nanochat/tokenizer.py
       and rustbpe/tokenizer.rs, including BPE logic and vocabulary loading.

    Example 2:
    Q: How is the model trained?
```

```
    A: Describe the training pipeline defined in train.py including
       optimizer, dataset loader, and training loop.

    Now rewrite this query to be more explicit:
    Q: {original_query}
    A:
    """
    response = llm.call(prompt)   # use your chosen LLM API
    return response.strip()
```

Rule-Based Rewriting Example (Python Pseudocode)

```
def rule_rewrite(q):
    q_lower = q.lower()
    if "tokenizer" in q_lower:
        return ("Explain the tokenizer implementation in "
                "nanochat/tokenizer.py and rustbpe/tokenizer.rs, "
                "including BPE logic and vocabulary loading.")
    if "train" in q_lower or "trained" in q_lower:
        return ("Describe the training pipeline in train.py: "
                "dataset loading, optimizer (AdamW) setup, and loop.")
    # fallback: return original
    return q
```

Before and After Retrieval Illustration

Before Rewriting	**After Rewriting**
Query: "How is training handled?"	Query: "Describe the functions and configuration used for model training in `train.py` and `adamw.py`, including optimizer setup and data loop."
Top results retrieved: Miscellaneous files (logging, configs).	Top results retrieved: `train.py`, `adamw.py`, `dataset.py` (high precision).
Answer: General statement on "training process."	Answer: Detailed explanation of dataset loading, optimizer, and training loop with code citations.

End of Document
